VeritasAI — Complete Project Master Plan
Multi-LLM Truth Engine with Hallucination Detection
Version 1.0 — Full Development Specification

Table of Contents

---

1. Project Overview

Name: VeritasAI (Veritas = Latin for "truth")
Tagline: "Ask anything. We ask everyone."
Deployment: Streamlit Community Cloud (free)
Purpose: A personal multi-LLM truth engine that queries multiple AI models simultaneously, detects hallucinations using semantic clustering + peer review, and returns a synthesized, confidence-scored answer to the user.

What makes it different from LLM Council and Ithy:
- Mathematical hallucination detection (DBSCAN + cosine similarity on embeddings) — not just social voting
- Per-model trust score visible to user
- Anomized peer review stage (models critique each other without knowing who wrote what)

- Mathematical hallucination detection (DBSCAN + cosine similarity on embeddings) — not just social voting
- Per-model trust score visible to user
- Anomized peer review stage (models critique each other without knowing who wrote what)
- Dual-signal truth scoring: mathematical (60%) + social peer rank (40%)
- Question enhancer rewrites raw input to optimal prompt before dispatching
- Free-tier only — runs at zero cost using 7+ provider free tiers
- Deployable publicly on Streamlit Cloud

---

2. Core Concept

The problem: Single LLMs hallucinate. No single model is always right.

The solution: Query 7 LLMs in parallel. Use two independent signals to find truth:
1. **Mathematical signal:** Convert all responses to semantic embeddings, cluster them with DBSCAN. The largest cluster = consensus. Models outside the cluster are flagged as potential hallucinations.
2. **Social signal:** Each model anonymously reviews all other responses and ranks them 1-to-N. Aggregate peer rank score computed per model.

Final trust score per model:
```
trust_score = (semantic_score × 0.6) + (peer_rank_score × 0.4)
```

Hallucination flag logic:
```
is_hallucinating = trust_score < 0.45 AND is_semantic_outlier
```
Both signals must agree. This reduces false positives.

Output: A synthesized consensus answer with a confidence percentage, per-model cards showing each response + trust score, and a hallucination warning if consensus is too low.

---

3. Architecture — 5 Stages

Every user query passes through exactly 5 stages in sequence.

```
### Stage 1 — Question Enhancer
- **File:** `agents/enhancer.py`
- **Input:** Raw user query (text, or transcribed speech, or image caption)
- **Process:** Sanitize input → call Groq llama-3.1-8b-instant with system prompt to rewrite query into an optimal, detailed, specific prompt. Detect query type: factual / analytical / creative / code / medical.
- **Output:** `EnhancedQuery(original: str, enhanced: str, query_type: str)`
- **API calls:** 1 (Groq free tier)

### Stage 2 — Parallel LLM Dispatch
- **File:** `core/dispatcher.py`
- **Input:** Enhanced prompt + optional image (base64) or context text
- **Process:** Fire all 7 enabled models simultaneously using `asyncio.gather()`. Each call wrapped in full exception handler. Each model has its own timeout, retry logic, and rate limit tracker.
- **Output:** `List[LLMResult]` — one per model, all statuses included (success/failed/skipped)
- **API calls:** 7 (all parallel — total wall time = slowest model, not sum)

### Stage 3 — Anonymized Peer Review
- **File:** `core/peer_review.py`
- **Input:** `List[LLMResult]` where status == SUCCESS
- **Process:** Shuffle responses, label them A/B/C/D... (anonymized). Each model receives all other responses in one prompt and ranks them 1-to-N. Parse "FINAL RANKING:" section from each response. Compute `peer_rank_score` per model (normalized 0-1).
- **Output:** `List[PeerReviewResult(model, raw_text, parsed_ranking, peer_rank_score)]`
- **API calls:** N more (same models, all parallel — each reviews all responses in one call, not N×N)

### Stage 4 — Dual-Signal Hallucination Detector
- **File:** `core/detector.py`
- **Input:** Successful LLMResults + PeerReviewResults
- **Process:**
  1. Embed all response texts with `sentence-transformers` (`all-MiniLM-L6-v2`)
  2. Compute pairwise cosine similarity matrix
  3. DBSCAN cluster (eps=0.25, min_samples=2, metric='cosine')
  4. Consensus cluster = largest cluster. Outliers = not in consensus cluster.
  5. `semantic_score` per model = cosine similarity to consensus centroid
  6. Blend: `trust_score = semantic_score × 0.6 + peer_rank_score × 0.4`
  7. Flag if `trust_score < 0.45` AND `is_semantic_outlier`
  8. Compute `consensus_ratio = len(trusted) / len(successful)`
- **Output:** `DetectionResult(trusted, outliers, trust_scores, consensus_ratio)`
- **API calls:** 0 — runs fully local on CPU
```

```
### Stage 5 — Result Combiner
- **File:** `agents/combiner.py`
- **Input:** Only the trusted LLMResults
- **Process:**
  - If `consensus_ratio >= 0.5`: Send trusted responses to combiner LLM with synthesis prompt. Produce final merged answer.
  - If `consensus_ratio < 0.5`: Skip synthesis. Set warning flag. Show all individual responses.
- **Output:** `FinalOutput(answer, consensus_ratio, trust_scores, peer_rankings, flags, follow_ups)`
- **API calls:** 1 (Groq fastest available trusted model)

**Total API calls per query (7 models): 1 + 7 + 7 + 0 + 1 = 16 calls**
*(vs LLM Council's 31 calls for 5 models)*

---

## 4. Trusted Free LLM Providers

All providers listed have permanent free tiers requiring no credit card.

### Tier 1 — Primary (always enabled)

| # | Provider | Model | Requests/Day | Key Strength |
|---|---|---|---|---|
| 1 | Google AI Studio | gemini-2.5-flash | 1,500/day | Best general quality, 1M context, multimodal |
| 2 | Groq | llama-3.3-70b-versatile | 1,000/day | Fastest (315 tok/sec), best for enhancer + combiner |
| 3 | Cerebras | llama-3.3-70b | 1M tokens/day | Most generous daily budget, 2,600 tok/sec |

### Tier 2 — Secondary (enabled by default)

| # | Provider | Model | Limit | Key Strength |
|---|---|---|---|---|
| 4 | Mistral AI | mistral-small-latest | 1B tokens/month | European, strong reasoning, diverse training |
| 5 | OpenRouter | deepseek/deepseek-r1:free | 50-1000/day | DeepSeek R1 reasoning model, diverse |
| 6 | NVIDIA NIM | deepseek-ai/deepseek-v4-pro | 40 RPM | 131K context, no credit card |

### Tier 3 — Optional (enable for more diversity)

| # | Provider | Model | Limit | Key Strength |
|---|---|---|---|---|
| 7 | OpenRouter | meta-llama/llama-4-maverick:free | see above | Different model family via same key |
```

Tier 3 — Optional (enable for more diversity)

#	Provider	Model	Limit	Key Strength
7	OpenRouter	meta-llama/llama-4-maverick:free	see above	Different model family via same key
8	Cohere	command-r-plus	$5 trial credits	Citation-backed grounded responses
9	SambaNova	Meta Llama / Qwen	$5 trial credits	Access to large 405B models

Role Assignment
- **Enhancer (Stage 1):** Groq llama-3.1-8b-instant (fastest, cheapest quota)
- **Dispatch (Stage 2):** All 7 enabled models
- **Peer Review (Stage 3):** All successful models from Stage 2
- **Combiner (Stage 5):** Groq llama-3.3-70b or Gemini flash (fastest successful model)

Signup URLs
- Groq: console.groq.com
- Gemini: aistudio.google.com
- Cerebras: cloud.cerebras.ai
- Mistral: console.mistral.ai
- OpenRouter: openrouter.ai
- NVIDIA NIM: build.nvidia.com
- Cohere: dashboard.cohere.com
- SambaNova: cloud.sambanova.ai

---

5. Full Feature List

v1 — Build Now (First Release)

Core Pipeline
- [ ] Question enhancer agent (Groq, system prompt, query type detection)
- [ ] Input sanitizer (empty check, 2000 char cap, whitespace strip, basic PII stub)
- [ ] Async multi-LLM dispatcher (asyncio.gather, per-model timeout + retry)
- [ ] LLMResult dataclass (model, status, response, error_type, error_msg, latency_ms, tokens)
- [ ] Exception handler — all types: auth(401/403), rate_limit(429 + retry), timeout, network, bad_response, empty_response
- [ ] Anonymized peer review stage (shuffle, label A/B/C, parallel dispatch, parse FINAL RANKING)
- [ ] Hallucination detector (sentence-transformers, DBSCAN, cosine similarity, dual-signal blending)

```
### v1 — Build Now (First Release)

**Core Pipeline**
- [ ] Question enhancer agent (Groq, system prompt, query type detection)
- [ ] Input sanitizer (empty check, 2000 char cap, whitespace strip, basic PII stub)
- [ ] Async multi-LLM dispatcher (asyncio.gather, per-model timeout + retry)
- [ ] LLMResult dataclass (model, status, response, error_type, error_msg, latency_ms, tokens)
- [ ] Exception handler — all types: auth(401/403), rate_limit(429 + retry), timeout, network, bad_response, empty_response
- [ ] Anonymized peer review stage (shuffle, label A/B/C, parallel dispatch, parse FINAL RANKING)
- [ ] Hallucination detector (sentence-transformers, DBSCAN, cosine similarity, dual-signal blending)
- [ ] Result combiner agent (synthesis prompt, low-consensus path)
- [ ] LLMAdapter abstract base class (3 methods: call, supports_images, get_model_id)
- [ ] All 7 LLM adapters (Groq, Gemini, Cerebras, Mistral, OpenRouter, NVIDIA NIM, Cohere)
- [ ] Semantic session cache (hash-based, same question = skip API calls)
- [ ] Rate limit tracker (per-model daily quota counter, JSON file)
- [ ] Model health config (enabled/disabled flag per model in config.yaml)
- [ ] Structured JSON logger (RotatingFileHandler + SQLite, all events)

**Input Modalities**
- [ ] Text input (st.text_area, 2000 char limit with counter)
- [ ] Voice input — mic button (audio_recorder_streamlit → Groq Whisper API)
- [ ] Image upload (st.file_uploader, JPG/PNG, base64 to Gemini)
- [ ] Webcam capture (st.camera_input → base64 to Gemini)

**UI Components**
- [ ] Landing state (logo + tagline + single search bar, nothing else)
- [ ] Enhanced query display (original query muted, enhanced query primary color)
- [ ] Live model status row (spinner → checkmark/X per model as they respond)
- [ ] Consensus answer card (large, confidence % badge, "Based on N/M models")
- [ ] Hallucination warning banner (amber, shown when consensus_ratio < 0.5)
- [ ] Per-model result cards (collapsed expanders, trust score bar, latency, token count)
- [ ] Trust score bar chart (st.bar_chart, green/amber/red by threshold)
- [ ] Query summary footer ("N models · M consensus · X flagged · Ys total")
- [ ] Debate mode toggle (show/hide raw peer review critique text)
- [ ] Fast mode / deep mode toggle (3 models no review vs 7 + review)
- [ ] Text-to-speech button (browser SpeechSynthesis API via st.components.html)
- [ ] Copy button on consensus answer
- [ ] Follow-up question suggestions (1 extra Groq call after synthesis)
- [ ] Feedback thumbs up/down (stored in SQLite feedback table)
```

- [ ] Consensus answer card (large, confidence % badge, "Based on N/M models")
- [ ] Hallucination warning banner (amber, shown when consensus_ratio < 0.5)
- [ ] Per-model result cards (collapsed expanders, trust score bar, latency, token count)
- [ ] Trust score bar chart (st.bar_chart, green/amber/red by threshold)
- [ ] Query summary footer ("N models · M consensus · X flagged · Ys total")
- [ ] Debate mode toggle (show/hide raw peer review critique text)
- [ ] Fast mode / deep mode toggle (3 models no review vs 7 + review)
- [ ] Text-to-speech button (browser SpeechSynthesis API via st.components.html)
- [ ] Copy button on consensus answer
- [ ] Follow-up question suggestions (1 extra Groq call after synthesis)
- [ ] Feedback thumbs up/down (stored in SQLite feedback table)
- [ ] Query history sidebar (last 20 queries, clickable to reload)
- [ ] Mobile-responsive layout (test mic + camera on iOS/Android)
- [ ] Dark/light mode (Streamlit config.toml)

Reliability & Deployment
- [ ] config.yaml (all settings, model list, thresholds, feature flags)
- [ ] .env.example (all API keys + DB_ENCRYPTION_KEY + ADMIN_PASSWORD_HASH)
- [ ] requirements.txt with pinned versions
- [ ] Streamlit secrets config (st.secrets replaces .env on cloud)
- [ ] Low-consensus handler (warn user, show all individual responses)
- [ ] All-models-fail handler (friendly message, no crash)
- [ ] Very short response filter (< 20 words excluded from clustering)
- [ ] README.md (setup, API key guide, local run, Streamlit deploy)

Observability
- [ ] Admin dashboard page (pages/admin.py, password-gated)
- [ ] Log viewer (filterable by level/model/event)
- [ ] Usage metrics cards (total queries, avg confidence, cache hit rate, model uptime)
- [ ] Per-model accuracy chart over time

v2 — Second Sprint

- [ ] PDF upload + question answering (pdfplumber, 3000 token injection)
- [ ] URL paste + article fact-check (trafilatura fetch + inject)
- [ ] CSV upload + data analysis questions (pandas, df.head(50).to_markdown())
- [ ] Claim-level confidence highlighting (spacy sentence split, per-sentence color)
- [ ] Multilingual auto-detect + translation (langdetect, translate query + answer)
- [ ] Export results to PDF and Markdown (fpdf2, st.download_button)

- [ ] PDF upload + question answering (pdfplumber, 3000 token injection)
- [ ] URL paste + article fact-check (trafilatura fetch + inject)
- [ ] CSV upload + data analysis questions (pandas, df.head(50).to_markdown())
- [ ] Claim-level confidence highlighting (spacy sentence split, per-sentence color)
- [ ] Multilingual auto-detect + translation (langdetect, translate query + answer)
- [ ] Export results to PDF and Markdown (fpdf2, st.download_button)
- [ ] Fact-check verdict mode (TRUE / FALSE / UNVERIFIABLE per claim)
- [ ] Topic mode presets (Science / Code / Medical / Legal / History)
- [ ] Share link via URL params (st.query_params, result stored in SQLite)
- [ ] Screenshot paste Ctrl+V (JS clipboard listener, base64 to session state)
- [ ] Conversation context mode (st.session_state last 3 Q&A pairs)
- [ ] Saved answers / favourites (star icon, sidebar section)
- [ ] Better TTS voice (gTTS or ElevenLabs free tier)
- [ ] Dynamic model weighting (Bayesian update from feedback history)
- [ ] Semantic query cache with persistence (shelve, cross-session)

v3 — Future

- [ ] Real-time streaming responses (each model streams tokens via st.write_stream)
- [ ] Local mode via Ollama (LLMAdapter for local Llama 3 — zero data leaves machine)
- [ ] Browser extension (Chrome Manifest v3 — highlight text, verify with VeritasAI)
- [ ] PWA / installable mobile app (Streamlit PWA wrapper)
- [ ] Public REST API (FastAPI backend, /query /verify /compare endpoints)
- [ ] Python SDK (pip install veritasai)
- [ ] Knowledge graph of verified facts (NetworkX, grows with every query)
- [ ] Temporal fact TTL (auto-expire time-sensitive verified facts)

---

6. File & Folder Structure

```
veritasai/
├── app.py                  ← Streamlit entry point, session state, page router
├── config.yaml             ← All settings: models, thresholds, feature flags
├── .env                    ← Real API keys (gitignored, never committed)
├── .env.example            ← Template with placeholder values only
├── requirements.txt        ← All deps pinned to exact versions
├── README.md               ← Setup, API key guide, deploy instructions
```

```
veritasai/
├── app.py                      ← Streamlit entry point, session state, page router
├── config.yaml                 ← All settings: models, thresholds, feature flags
├── .env                        ← Real API keys (gitignored, never committed)
├── .env.example                ← Template with placeholder values only
├── requirements.txt            ← All deps pinned to exact versions
├── README.md                   ← Setup, API key guide, deploy instructions
├── .gitignore                  ← Must include .env, logs/, *.db
│
├── core/                       ← Pure Python, zero Streamlit dependency
│   ├── adapter.py              ← Abstract LLMAdapter base class
│   ├── result.py               ← LLMResult, LLMStatus, EnhancedQuery, FinalOutput dataclasses
│   ├── dispatcher.py           ← asyncio fan-out, gather, per-model timeout
│   ├── peer_review.py          ← Anonymize, dispatch review calls, parse rankings
│   ├── detector.py             ← Hallucination detection (embeddings + DBSCAN + trust scoring)
│   ├── combiner_logic.py       ← Synthesis prompt building, low-consensus logic
│   ├── cache.py                ← Session cache (dict) + persistent cache (shelve)
│   ├── sanitizer.py            ← Input cleaning, length cap, basic PII stub
│   ├── rate_tracker.py         ← Per-model daily quota tracking (JSON file)
│   └── logger.py               ← RotatingFileHandler + SQLite structured event logger
│
├── agents/                     ← One file per LLM adapter + special agents
│   ├── enhancer.py             ← Question enhancer agent (Groq fast model)
│   ├── combiner.py             ← Result synthesis agent (fastest trusted model)
│   ├── groq_adapter.py         ← Groq LLMAdapter implementation
│   ├── gemini_adapter.py       ← Gemini LLMAdapter (supports images)
│   ├── cerebras_adapter.py     ← Cerebras LLMAdapter
│   ├── mistral_adapter.py      ← Mistral LLMAdapter
│   ├── openrouter_adapter.py   ← OpenRouter LLMAdapter (handles :free model suffix)
│   ├── nvidia_nim_adapter.py   ← NVIDIA NIM LLMAdapter
│   └── cohere_adapter.py       ← Cohere LLMAdapter
│
├── ui/                         ← All Streamlit components, isolated and reusable
│   ├── search_bar.py           ← Text input + voice mic button + image upload
│   ├── enhanced_query.py       ← Show original vs enhanced query diff
│   ├── model_status.py         ← Live status row (spinner/check/X per model)
│   ├── consensus_card.py       ← Hero answer card with confidence badge
│   ├── warning_banner.py       ← Amber warning for low consensus
```

```
    └── cohere_adapter.py          ← Cohere LLMAdapter

├── ui/                            ← All Streamlit components, isolated and reusable
    ├── search_bar.py              ← Text input + voice mic button + image upload
    ├── enhanced_query.py          ← Show original vs enhanced query diff
    ├── model_status.py            ← Live status row (spinner/check/X per model)
    ├── consensus_card.py          ← Hero answer card with confidence badge
    ├── warning_banner.py          ← Amber warning for low consensus
    ├── model_card.py              ← Per-model expander: trust bar, response, latency
    ├── trust_chart.py             ← Horizontal bar chart of trust scores
    ├── feedback.py                ← Thumbs up/down + write to SQLite
    ├── summary_footer.py          ← "N models · M consensus · Xs total"
    ├── sidebar.py                 ← Query history + mode toggles + settings
    └── tts.py                     ← Browser SpeechSynthesis JS component

├── pages/                         ← Streamlit multipage
    └── admin.py                   ← Log viewer, usage charts, model health, password-gated

├── logs/                          ← Gitignored directory
    ├── veritasai.log              ← Rotating flat log (10MB max, 5 backups)
    └── admin.db                   ← SQLite encrypted database (sqlcipher)

└── tests/                         ← Unit tests
    ├── test_detector.py
    ├── test_dispatcher.py
    ├── test_peer_review.py
    └── test_sanitizer.py
```

---

7. Every File Explained

`core/adapter.py` — The Abstract Interface

```python
from abc import ABC, abstractmethod
from core.result import LLMResult

class LLMAdapter(ABC):
```

```
## 7. Every File Explained

### `core/adapter.py` — The Abstract Interface
```python
from abc import ABC, abstractmethod
from core.result import LLMResult

class LLMAdapter(ABC):
    @abstractmethod
    async def call(self, prompt: str, image_b64: str | None = None) -> LLMResult:
        """Make a single API call. Never raises — returns LLMResult with error info."""
        pass

    @abstractmethod
    def supports_images(self) -> bool:
        """Return True if this adapter handles image inputs."""
        pass

    @abstractmethod
    def get_model_id(self) -> str:
        """Return the model identifier string."""
        pass

    @property
    @abstractmethod
    def name(self) -> str:
        """Human-readable name for display."""
        pass
```

`core/result.py` — All Data Models
```python
from dataclasses import dataclass, field
from enum import Enum

class LLMStatus(Enum):
    SUCCESS = "success"
    FAILED = "failed"
```
```

```python
class LLMStatus(Enum):
    SUCCESS = "success"
    FAILED = "failed"
    SKIPPED = "skipped"

@dataclass
class LLMResult:
    model: str
    status: LLMStatus
    response: str | None
    error_type: str | None
    error_msg: str | None
    latency_ms: int
    tokens_used: int = 0
    peer_rank_score: float = 0.5   # filled after peer review stage
    trust_score: float = 0.0       # filled after detection stage
    is_outlier: bool = False        # filled after detection stage

@dataclass
class EnhancedQuery:
    original: str
    enhanced: str
    query_type: str   # factual | analytical | creative | code | medical

@dataclass
class PeerReviewResult:
    model: str
    raw_ranking_text: str
    parsed_ranking: list[str]   # ordered list of labels e.g. ['B','A','C','D']
    peer_rank_score: float      # 0-1, higher = ranked better by peers

@dataclass
class DetectionResult:
    trusted: list[LLMResult]
    outliers: list[LLMResult]
    trust_scores: dict[str, float]
    consensus_ratio: float
    low_consensus: bool         # True when consensus_ratio < 0.5
```

```
    low_consensus: bool         # True when consensus_ratio < 0.5

@dataclass
class FinalOutput:
    answer: str | None
    consensus_ratio: float
    trust_scores: dict[str, float]
    peer_rankings: dict[str, float]
    hallucination_flags: list[str]   # list of model names flagged
    follow_up_questions: list[str]
    low_consensus: bool
    all_results: list[LLMResult]
    total_latency_ms: int
```

`core/dispatcher.py` — Async Fan-out

```python
import asyncio
import time

async def dispatch_all(
    adapters: list,
    prompt: str,
    image_b64: str | None = None,
    timeout_s: int = 30
) -> list[LLMResult]:
    """Fire all adapters in parallel. Returns all results including failures."""

    async def call_with_timeout(adapter):
        try:
            return await asyncio.wait_for(
                adapter.call(prompt, image_b64),
                timeout=timeout_s
            )
        except asyncio.TimeoutError:
            return LLMResult(
                model=adapter.name,
                status=LLMStatus.FAILED,
                response=None,
```

```
        )
    except asyncio.TimeoutError:
        return LLMResult(
            model=adapter.name,
            status=LLMStatus.FAILED,
            response=None,
            error_type="timeout",
            error_msg=f"Exceeded {timeout_s}s",
            latency_ms=timeout_s * 1000
        )

results = await asyncio.gather(
    *[call_with_timeout(a) for a in adapters],
    return_exceptions=False
)
return list(results)
```

`core/peer_review.py` — Anonymized Ranking

```python
REVIEW_PROMPT = """You are evaluating AI responses. The question was:
"{question}"

Below are {n} responses labeled A through {last_label}:
{responses}

Evaluate each response for accuracy, completeness, and clarity.
Respond ONLY with the following format — nothing else:
FINAL RANKING:
1. Response [letter]
2. Response [letter]
...
(Best first, worst last. Every letter must appear exactly once.)"""

# Shuffle responses before labeling to prevent position bias
# Each model reviews ALL responses in ONE call (not N×N like LLM Council)
# parse_ranking() extracts the ordered letter list from "FINAL RANKING:" section
# compute_peer_score() normalizes rank position to 0-1 score
```

```
# Shuffle responses before labeling to prevent position bias
# Each model reviews ALL responses in ONE call (not N×N like LLM Council)
# parse_ranking() extracts the ordered letter list from "FINAL RANKING:" section
# compute_peer_score() normalizes rank position to 0-1 score
```

`core/detector.py` — Hallucination Detection

```python
# sentence-transformers: all-MiniLM-L6-v2 (80MB, runs on CPU, loads once)
# DBSCAN params: eps=0.25, min_samples=2, metric='cosine'
# trust_score = semantic_score × 0.6 + peer_rank_score × 0.4
# is_hallucinating = trust_score < 0.45 AND is_semantic_outlier (both signals must agree)
# consensus_ratio = len(trusted) / len(total_successful)
```

Exception Handler Pattern (used in every adapter)

```python
async def call(self, prompt: str, image_b64=None) -> LLMResult:
    start = time.time()
    for attempt in range(self.max_retries):
        try:
            response = await self._api_call(prompt, image_b64)

            if not response or len(response.strip()) < 20:
                return LLMResult(self.name, LLMStatus.FAILED, None,
                    "empty_response", "Response too short", elapsed())

            return LLMResult(self.name, LLMStatus.SUCCESS, response,
                None, None, elapsed())

        except AuthenticationError:
            return LLMResult(self.name, LLMStatus.FAILED, None,
                "auth", "Invalid API key", elapsed())

        except RateLimitError:
            if attempt < self.max_retries - 1:
                await asyncio.sleep(2 ** attempt)   # 1s, 2s exponential backoff
                continue
            return LLMResult(self.name, LLMStatus.FAILED, None,
```

```
        except RateLimitError:
            if attempt < self.max_retries - 1:
                await asyncio.sleep(2 ** attempt)   # 1s, 2s exponential backoff
                continue
            return LLMResult(self.name, LLMStatus.FAILED, None,
                "rate_limit", "Rate limit exceeded after retries", elapsed())

        except asyncio.TimeoutError:
            return LLMResult(self.name, LLMStatus.FAILED, None,
                "timeout", f"Exceeded {self.timeout}s", elapsed())

        except (ConnectionError, SSLError):
            return LLMResult(self.name, LLMStatus.FAILED, None,
                "network", "Network/SSL error", elapsed())

        except Exception as e:
            return LLMResult(self.name, LLMStatus.FAILED, None,
                type(e).__name__, str(e), elapsed())

    # This line is never reached but satisfies type checker
    return LLMResult(self.name, LLMStatus.FAILED, None, "unknown", "Max retries", elapsed())
```

---

8. Data Models & Schemas

Admin SQLite Tables

```sql
-- Every event the system produces
CREATE TABLE logs (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    ts          TEXT NOT NULL,
    level       TEXT NOT NULL,    -- INFO | WARN | ERROR | OK | DEBUG
    session_id  TEXT NOT NULL,
    event       TEXT NOT NULL,    -- llm_call_success | hallucination_flagged | etc
    model       TEXT,
    query_hash  TEXT,             -- SHA-256 of enhanced prompt
```

```
-- Every event the system produces
CREATE TABLE logs (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    ts           TEXT NOT NULL,
    level        TEXT NOT NULL,   -- INFO | WARN | ERROR | OK | DEBUG
    session_id   TEXT NOT NULL,
    event        TEXT NOT NULL,   -- llm_call_success | hallucination_flagged | etc
    model        TEXT,
    query_hash   TEXT,            -- SHA-256 of enhanced prompt
    latency_ms   INTEGER,
    trust_score  REAL,
    consensus_ratio REAL,
    error_type   TEXT,
    error_msg    TEXT,
    tokens_used  INTEGER,
    hallucination_flagged INTEGER DEFAULT 0,
    cache_hit    INTEGER DEFAULT 0,
    component    TEXT             -- enhancer | dispatcher | detector | combiner | ui
);

-- Every query submitted
CREATE TABLE queries (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    ts              TEXT NOT NULL,
    session_id      TEXT NOT NULL,
    query_hash      TEXT NOT NULL,
    original_query  TEXT NOT NULL,
    enhanced_query  TEXT NOT NULL,
    query_type      TEXT,
    consensus_ratio REAL,
    final_answer    TEXT,
    total_latency_ms INTEGER,
    models_used     TEXT,         -- JSON array of model names
    models_trusted  TEXT,         -- JSON array
    models_flagged  TEXT          -- JSON array
);

-- Per-model responses for every query
CREATE TABLE model_responses (
```

```
-- Per-model responses for every query
CREATE TABLE model_responses (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    query_hash   TEXT NOT NULL,
    model        TEXT NOT NULL,
    response     TEXT,
    trust_score  REAL,
    peer_score   REAL,
    is_outlier   INTEGER DEFAULT 0,
    latency_ms   INTEGER,
    tokens_used  INTEGER,
    status       TEXT,
    error_type   TEXT
);

-- User feedback
CREATE TABLE feedback (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    ts         TEXT NOT NULL,
    session_id TEXT NOT NULL,
    query_hash TEXT NOT NULL,
    vote       TEXT NOT NULL,    -- up | down
    comment    TEXT
);
```

Log Event Types

```
app_start | app_error | config_loaded
query_received | query_enhanced | cache_hit | cache_miss
llm_dispatch_start | llm_call_success | llm_call_failed | llm_call_timeout
rate_limit_hit | rate_limit_retry | short_response_filtered
peer_review_start | peer_review_complete | peer_review_parse_failed
consensus_computed | hallucination_flagged | low_consensus_warning
synthesis_complete | query_complete
user_feedback | admin_access
```

```
---

## 9. Security Architecture

### Layer 1 — API Key Protection
- All keys live in `.env` locally and `st.secrets` on Streamlit Cloud
- `.env` listed in `.gitignore` from day one — never committed
- Logger never writes key values. Logs `groq_key_set: true` only
- `detect-secrets` pre-commit hook blocks any commit containing key-like strings
- Install: `pip install detect-secrets && detect-secrets scan > .secrets.baseline`

### Layer 2 — Database Encryption
- `sqlcipher` library provides AES-256 encryption of the entire `admin.db` file at rest
- Key derived from `DB_ENCRYPTION_KEY` in `.env`
- Install: `pip install sqlcipher3`
- File permissions: `chmod 600 logs/admin.db logs/veritasai.log`

### Layer 3 — Query Privacy
- Query text stored encrypted in SQLite. In flat logs, stored as `SHA-256(query)` only
- PII scrubber runs before any LLM dispatch (v1: regex for email/phone, v2: spacy NER)
- Each provider's privacy policy confirmed: Groq, Gemini, Mistral, Cerebras do NOT train on API data

### Layer 4 — Application Security
- Admin dashboard password: stored as `bcrypt` hash in `st.secrets`, never plaintext
- Input injection guard: all user text passed through template function before prompt insertion
- Session isolation: each Streamlit session gets a UUID. One user cannot see another's results
- Rate limiting: max 10 queries per hour per session (in-memory counter)

### Pre-deployment Checklist
```

```
[ ] .env in .gitignore
[ ] DB_ENCRYPTION_KEY is 32+ random characters
[ ] ADMIN_PASSWORD_HASH is a bcrypt hash (not plaintext)
[ ] detect-secrets pre-commit hook installed
[ ] logs/ in .gitignore
[ ] No API keys in any Python files
[ ] .env.example has placeholder values only
[ ] chmod 600 logs/admin.db
```

10. UI Architecture

Three States

State A — Landing (no query yet)
- Header bar (logo left, admin link right, height 48px)
- Centered hero zone: tagline + large search bar (48px height) + input mode buttons (text/mic/image)
- Footer: model count badge + GitHub link
- Nothing else. Zero clutter. (Hick's Law)

State B — Query Running
- Header + compact search bar showing enhanced query in muted text
- Enhanced query display (original → enhanced diff)
- Live model status row (5-7 model pills, spinner → checkmark/X)
- Result cards appear as each model finishes (don't wait for all)

State C — Results Complete
- Header + compact search (new query possible)
- Consensus answer card (large, prominent, confidence badge)
- Hallucination warning banner (amber, conditional on consensus_ratio < 0.5)
- Trust score bar chart
- Per-model cards (collapsed grid, expand to read full response)
- Follow-up question buttons
- Feedback buttons
- Summary footer ("N models · M consensus · Xs total")

Psychology Principles Applied
1. **Hick's Law:** Landing shows only the search bar. Nothing competes for attention.
2. **Anxiety elimination:** Live model status updates convert waiting anxiety into anticipation.
3. **Anchoring:** Consensus answer is always first, largest, most prominent.
4. **Trust through transparency:** Every model's response visible. Nothing hidden.
5. **Color semantics:** Green = trusted. Amber = uncertain. Gray = failed. Never decorative.
6. **Progressive disclosure:** Model cards collapsed by default. Details on click.
7. **Peak-end rule:** Summary footer gives the experience a satisfying conclusion.
8. **White space = intelligence:** Minimum 24px padding inside cards. Nothing feels cramped.

---

11. Exception Handling

Every LLM call returns `LLMResult`. Never raises. Typed error categories:

Error Type	Trigger	Retry?	Behavior
`auth`	401, 403, invalid key	No	Fail immediately
`rate_limit`	429, quota exceeded	Yes (2×, backoff 1s/2s)	Retry with exponential backoff
`timeout`	asyncio.TimeoutError	No	Fail immediately
`network`	DNS, SSL, ConnectionError	No	Fail immediately
`server_error`	500, 502, 503, 504	Yes (1×, after 2s)	Single retry
`empty_response`	Response < 20 words	No	Filter before clustering
`bad_json`	Malformed API response	No	Fail immediately
`content_filtered`	Safety block / refusal	No	Mark as filtered
`token_limit`	400, context too long	No	Fail, truncate input and retry (v2)
`model_unavailable`	Model deprecated / overloaded	No	Fail, mark model unhealthy

Minimum threshold: After all retries, if `len(successful_results) < 2`, skip detector + combiner. Show partial results with a warning. If `len(successful_results) == 0`, show friendly error: "All models are currently unavailable. Please try again in a few minutes."

---

12. Admin Log System

Log Entry Schema (JSON per line in flat file, row in SQLite)

```json
{
  "ts": "2026-06-06T14:32:01.443Z",
  "level": "INFO",
  "session_id": "uuid-v4",
  "event": "llm_call_success",
  "model": "groq",
  "query_hash": "sha256-of-enhanced-prompt",
  "latency_ms": 1243,
  "trust_score": 0.91,
  "consensus_ratio": 0.75,
  "error_type": null,
```

```
  "ts": "2026-06-06T14:32:01.443Z",
  "level": "INFO",
  "session_id": "uuid-v4",
  "event": "llm_call_success",
  "model": "groq",
  "query_hash": "sha256-of-enhanced-prompt",
  "latency_ms": 1243,
  "trust_score": 0.91,
  "consensus_ratio": 0.75,
  "error_type": null,
  "error_msg": null,
  "tokens_used": 512,
  "hallucination_flagged": false,
  "cache_hit": false,
  "component": "dispatcher"
}
```

Storage Strategy
- **Local development:** Python `logging.handlers.RotatingFileHandler` (10MB max, 5 backup files) for flat log. SQLite (with sqlcipher) for queryable history.
- **Streamlit Cloud:** Same flat log. SQLite file persists in Streamlit's `/tmp` — note: clears on redeploy. For persistent cloud logs, write to an external SQLite host or simple JSON append to Streamlit's file system.

Admin Dashboard (pages/admin.py)
- Password-gated via `st.secrets["admin_password_hash"]` + bcrypt comparison
- 4 metric cards: total queries, avg confidence %, cache hit rate, model uptime %
- Log viewer table with filters by level, model, event type, date range
- Per-model accuracy chart over time (line chart)
- Model health table (enabled/disabled, last seen, failure rate)

---

13. Config Reference

config.yaml — Complete Structure
```yaml
app:
  name: VeritasAI
```

```
## 13. Config Reference

### config.yaml — Complete Structure
```yaml
app:
  name: VeritasAI
  version: 1.0.0
  max_input_chars: 2000
  session_rate_limit: 10        # queries per hour per session
  min_responses_required: 2     # minimum successful responses to proceed

pipeline:
  enhancer_model: groq          # which adapter to use for enhancement
  combiner_model: auto          # 'auto' = fastest successful model from stage 2
  consensus_threshold: 0.5      # below this = low consensus warning
  trust_threshold: 0.45         # below this = hallucination flag
  semantic_weight: 0.6          # weight for mathematical signal
  peer_weight: 0.4              # weight for social peer ranking signal

detector:
  eps: 0.25                     # DBSCAN cluster distance threshold
  min_samples: 2                # minimum models to form a cluster
  embedding_model: all-MiniLM-L6-v2
  short_response_min_words: 20  # responses shorter than this are filtered

models:
  - name: groq
    model_id: llama-3.3-70b-versatile
    enhancer_model_id: llama-3.1-8b-instant
    enabled: true
    daily_limit: 100
    timeout_s: 30
    max_retries: 2
    supports_images: false
    roles: [enhancer, dispatch, review, combine]

  - name: gemini
    model_id: gemini-2.5-flash
```
```

```
    roles: [enhancer, dispatch, review, combine]

  - name: gemini
    model_id: gemini-2.5-flash
    enabled: true
    daily_limit: 200
    timeout_s: 30
    max_retries: 2
    supports_images: true
    roles: [dispatch, review, combine]

  - name: cerebras
    model_id: llama-3.3-70b
    enabled: true
    daily_limit: 300
    timeout_s: 20
    max_retries: 1
    supports_images: false
    roles: [dispatch, review]

  - name: mistral
    model_id: mistral-small-latest
    enabled: true
    daily_limit: 50
    timeout_s: 40
    max_retries: 1
    supports_images: false
    roles: [dispatch, review]

  - name: openrouter_deepseek
    model_id: deepseek/deepseek-r1:free
    enabled: true
    daily_limit: 50
    timeout_s: 45
    max_retries: 1
    supports_images: false
    roles: [dispatch, review]

  - name: nvidia_nim
```

```
  - name: nvidia_nim
    model_id: deepseek-ai/deepseek-v4-pro
    enabled: true
    daily_limit: 80
    timeout_s: 30
    max_retries: 1
    supports_images: false
    roles: [dispatch, review]

  - name: openrouter_llama4
    model_id: meta-llama/llama-4-maverick:free
    enabled: true
    daily_limit: 50
    timeout_s: 40
    max_retries: 1
    supports_images: false
    roles: [dispatch, review]

features:
  voice_input: true
  image_input: true
  webcam_input: true
  tts_output: true
  follow_up_suggestions: true
  debate_mode: true
  fast_mode_available: true
  query_history: true
  feedback: true
  admin_dashboard: true
  cache_enabled: true

logging:
  level: INFO
  max_bytes: 10485760   # 10MB
  backup_count: 5
  db_encrypt: true
```

.env.example — Complete Template

.env.example — Complete Template

```bash
# VeritasAI — copy to .env, fill in your keys. NEVER commit .env
# Signup URLs in README.md

# LLM API Keys
GROQ_API_KEY=gsk_...
GEMINI_API_KEY=AIza...
CEREBRAS_API_KEY=csk-...
MISTRAL_API_KEY=...
OPENROUTER_API_KEY=sk-or-...
NVIDIA_NIM_API_KEY=nvapi-...
COHERE_API_KEY=...
SAMBANOVA_API_KEY=...

# Security
DB_ENCRYPTION_KEY=change-this-to-a-32-plus-character-random-string
ADMIN_PASSWORD_HASH=bcrypt-hash-of-your-admin-password
SESSION_SECRET=random-32-char-string
```

---

14. Build Order

Build in this exact sequence. Each step enables the next. Do not skip ahead.

Step	What to Build	Output Files	Why This Order
1	Project scaffold	all folders, config.yaml, .env.example, requirements.txt, README.md	Foundation for everything
2	Data models	core/result.py	Shape that all other files depend on
3	Admin logger	core/logger.py, logs/	Log from day one — all adapters use it
4	LLMAdapter interface	core/adapter.py	Interface before any implementation
5	Groq adapter	agents/groq_adapter.py	First adapter, use as template for others
6	Remaining adapters	agents/gemini_adapter.py, etc. (6 files)	Copy Groq pattern, adjust API client
7	Rate tracker + sanitizer	core/rate_tracker.py, core/sanitizer.py	Used by dispatcher
8	Async dispatcher	core/dispatcher.py	Fan-out logic
9	Question enhancer	agents/enhancer.py	Stage 1 complete

5	Groq adapter	agents/groq_adapter.py	First adapter, use as template for others
6	Remaining adapters	agents/gemini_adapter.py, etc. (6 files)	Copy Groq pattern, adjust API client
7	Rate tracker + sanitizer	core/rate_tracker.py, core/sanitizer.py	Used by dispatcher
8	Async dispatcher	core/dispatcher.py	Fan-out logic
9	Question enhancer	agents/enhancer.py	Stage 1 complete
10	Peer review	core/peer_review.py	Stage 3 logic
11	Hallucination detector	core/detector.py	Stage 4 logic (sentence-transformers)
12	Combiner	agents/combiner.py, core/combiner_logic.py	Stage 5 complete
13	Session cache	core/cache.py	Plugs into dispatcher
14	UI components	ui/ (all 11 files)	All components isolated
15	Main app	app.py	Wires all components together
16	Admin dashboard	pages/admin.py	Separate page
17	Tests	tests/ (4 files)	Verify core logic
18	Pin deps + deploy config	requirements.txt final, .streamlit/config.toml	Deploy-ready

---

15. Dependencies

requirements.txt (pinned)

```
# Streamlit
streamlit==1.45.0
streamlit-audio-recorder==0.0.8    # voice input mic button

# Async HTTP
aiohttp==3.9.5
aiofiles==23.2.1

# LLM Client Libraries
groq==0.11.0
google-generativeai==0.8.3
mistralai==1.0.3
cohere==5.5.3
openai==1.35.0    # used for OpenRouter + NVIDIA NIM (OpenAI-compatible endpoints)
cerebras_cloud_sdk==1.0.0

# ML / NLP
sentence-transformers==2.7.0
```

```
openai==1.35.0    # used for OpenRouter + NVIDIA NIM (OpenAI-compatible endpoints)
cerebras_cloud_sdk==1.0.0

# ML / NLP
sentence-transformers==2.7.0
scikit-learn==1.5.0
numpy==1.26.4

# Data
pandas==2.2.2
pyyaml==6.0.1

# Security
python-dotenv==1.0.1
bcrypt==4.1.3
sqlcipher3==0.5.3     # encrypted SQLite
cryptography==42.0.8

# Logging
colorlog==6.8.2

# Utils
httpx==0.27.0
langdetect==1.0.9
pyperclip==1.8.2

# Testing
pytest==8.2.2
pytest-asyncio==0.23.7
```

Python Version
Python 3.11+ required (for `str | None` union syntax and `asyncio` improvements)

Streamlit Cloud Constraints
- RAM limit: 1GB on free tier
- `sentence-transformers` with `all-MiniLM-L6-v2` uses ~400MB
- Streamlit itself uses ~200MB
- Remaining ~400MB for runtime — sufficient for v1

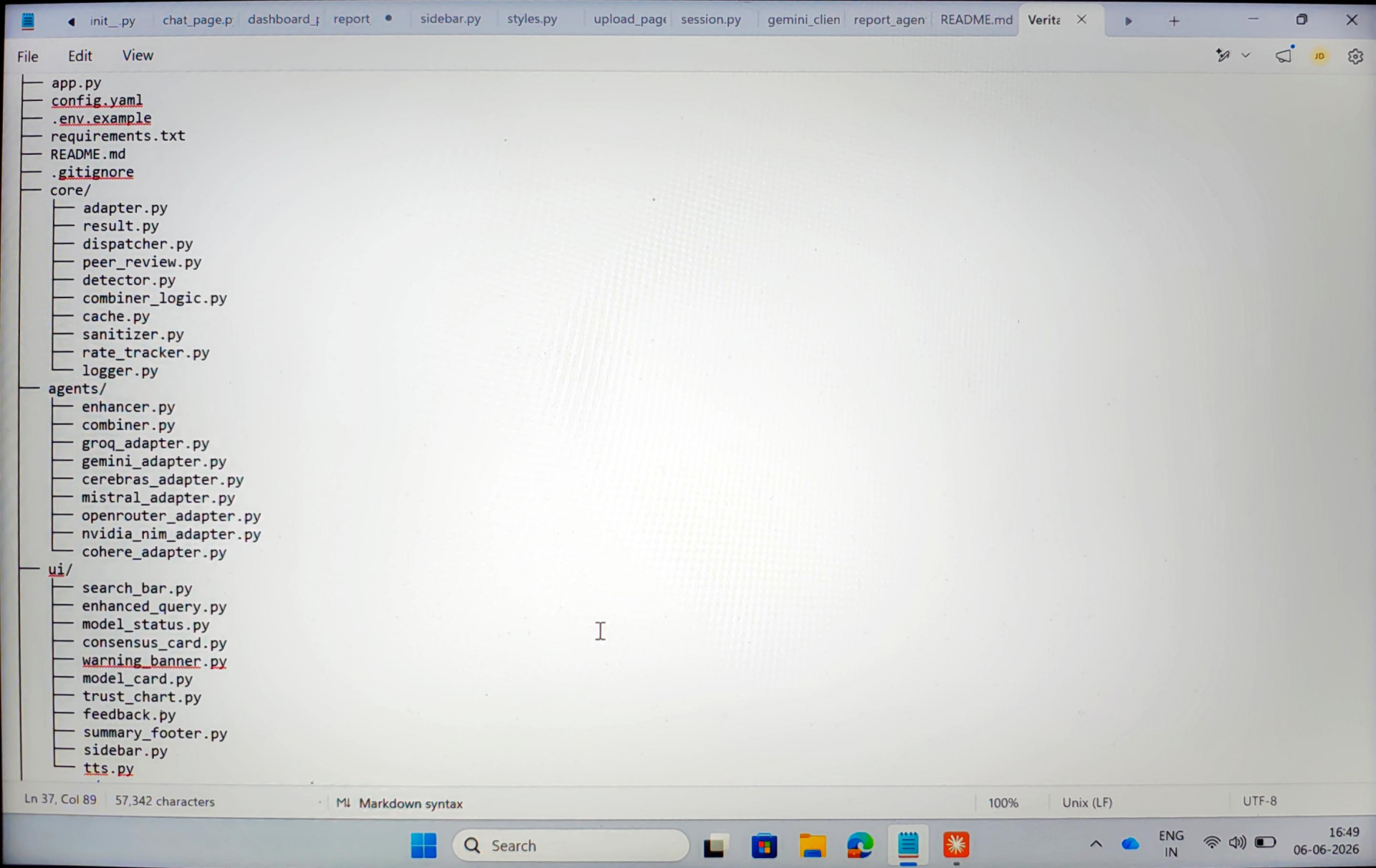
Streamlit Cloud Constraints
- RAM limit: 1GB on free tier
- `sentence-transformers` with `all-MiniLM-L6-v2` uses ~400MB
- Streamlit itself uses ~200MB
- Remaining ~400MB for runtime — sufficient for v1

---

16. AI Code Generation Prompt

> **Copy this entire prompt and give it to Claude, GPT-4, Cursor, or any AI coding tool to generate the complete project.**

```
You are a senior Python developer and ML engineer. Build a complete production-ready
Python project called VeritasAI — a multi-LLM truth engine with hallucination detection.

## PROJECT OVERVIEW

VeritasAI queries 7 large language models in parallel, uses mathematical semantic
clustering (DBSCAN + cosine similarity) AND anonymized peer review to detect which
models are hallucinating, and returns a synthesized, confidence-scored answer to the
user via a Streamlit web app. Deploy target: Streamlit Community Cloud, free tier.

## EXACT FILE STRUCTURE TO CREATE

Create every file listed below with complete, working, production-quality code:

veritasai/
├── app.py
├── config.yaml
├── .env.example
├── requirements.txt
├── README.md
├── .gitignore
├── core/
│   ├── adapter.py
│   ├── result.py
```


init__.py
chat_page.p
dashboard_
report
sidebar.py
styles.py
upload_page
session.py
gemini_clien
report_agen
README.md
Verita
File
Edit
View
├── app.py
├── config.yaml
├── .env.example
├── requirements.txt
├── README.md
├── .gitignore
├── core/
│   ├── adapter.py
│   ├── result.py
│   ├── dispatcher.py
│   ├── peer_review.py
│   ├── detector.py
│   ├── combiner_logic.py
│   ├── cache.py
│   ├── sanitizer.py
│   ├── rate_tracker.py
│   └── logger.py
├── agents/
│   ├── enhancer.py
│   ├── combiner.py
│   ├── groq_adapter.py
│   ├── gemini_adapter.py
│   ├── cerebras_adapter.py
│   ├── mistral_adapter.py
│   ├── openrouter_adapter.py
│   ├── nvidia_nim_adapter.py
│   └── cohere_adapter.py
├── ui/
│   ├── search_bar.py
│   ├── enhanced_query.py
│   ├── model_status.py
│   ├── consensus_card.py
│   ├── warning_banner.py
│   ├── model_card.py
│   ├── trust_chart.py
│   ├── feedback.py
│   ├── summary_footer.py
│   ├── sidebar.py
│   └── tts.py
Ln 37, Col 89
57,342 characters
Markdown syntax
100%
Unix (LF)
UTF-8
Search
ENG
IN
16:49
06-06-2026

```
    ├── trust_chart.py
    ├── feedback.py
    ├── summary_footer.py
    ├── sidebar.py
    └── tts.py
├── pages/
    └── admin.py
└── tests/
    ├── test_detector.py
    ├── test_dispatcher.py
    ├── test_peer_review.py
    └── test_sanitizer.py
```

PIPELINE — 5 STAGES

Every query flows through exactly these stages:

STAGE 1 — QUESTION ENHANCER (agents/enhancer.py)
- Input: raw user query string
- Run input sanitizer first: strip whitespace, reject empty, cap at 2000 chars
- Call Groq llama-3.1-8b-instant with this system prompt:
  "You are an expert prompt engineer. Rewrite the user's question to be maximally specific, detailed, and clear for a large language model. Preserve the user's intent exactly. Add relevant context, specify the type of answer expected, and clarify any ambiguities. Return only the rewritten question, nothing else."
- Detect query_type: factual | analytical | creative | code | medical
- Return: EnhancedQuery(original, enhanced, query_type)
- API calls: 1 (Groq)

STAGE 2 — PARALLEL LLM DISPATCH (core/dispatcher.py)
- Input: enhanced prompt string + optional image base64
- Fire all 7 enabled model adapters simultaneously with asyncio.gather()
- Each adapter wrapped with full exception handling (see Exception Handler section)
- Timeout per model from config.yaml
- Return: List[LLMResult] — all results including failures
- API calls: 7 parallel

STAGE 3 — ANONYMIZED PEER REVIEW (core/peer_review.py)
- Input: only the LLMResults where status == SUCCESS

```
- Return: List[LLMResult] — all results including failures
- API calls: 7 parallel

STAGE 3 — ANONYMIZED PEER REVIEW (core/peer_review.py)
- Input: only the LLMResults where status == SUCCESS
- If fewer than 2 successful: skip this stage, set all peer_rank_score = 0.5
- Shuffle the successful responses and assign labels: A, B, C, D, E...
  (shuffle prevents position bias — model responses in random order each time)
- Build review prompt:
  "You are evaluating AI responses. The question was: '{question}'

   Below are {n} responses labeled {labels}:
   {response_A_text}
   Response B: {response_B_text}
   ...

   Evaluate each response for factual accuracy, completeness, and clarity.
   Respond ONLY with:
   FINAL RANKING:
   1. Response [letter]
   2. Response [letter]
   (best first, worst last. Every letter must appear exactly once.)"
- Send this review prompt to ALL successful models simultaneously (asyncio.gather)
  NOTE: each model reviews ALL other responses in ONE call. This is N calls total,
  NOT N×N calls. This is key efficiency advantage over LLM Council.
- Parse "FINAL RANKING:" section from each response using regex
- For each model: compute peer_rank_score = 1 - (average_rank_position / (n-1))
  where rank position 0 = ranked first (best) and n-1 = ranked last (worst)
  Normalize so rank_score is 0 to 1 where 1 = always ranked first by peers
- Return: List[PeerReviewResult(model, raw_text, parsed_ranking, peer_rank_score)]
- API calls: N (same count as successful models from stage 2)

STAGE 4 — HALLUCINATION DETECTOR (core/detector.py)
- Input: List[LLMResult] (successful only) + List[PeerReviewResult]
- Uses sentence-transformers model: all-MiniLM-L6-v2 (load once at startup)
- Step 1: Embed all response texts: embeddings = model.encode(texts, normalize=True)
- Step 2: Run DBSCAN: eps=0.25, min_samples=2, metric='cosine'
- Step 3: Find consensus cluster = cluster with most members (label != -1)
       If no valid cluster: return all as trusted with trust_score = 0.5
```

```
- Uses sentence-transformers model: all-MiniLM-L6-v2 (load once at startup)
- Step 1: Embed all response texts: embeddings = model.encode(texts, normalize=True)
- Step 2: Run DBSCAN: eps=0.25, min_samples=2, metric='cosine'
- Step 3: Find consensus cluster = cluster with most members (label != -1)
        If no valid cluster: return all as trusted with trust_score = 0.5
- Step 4: For each model:
        semantic_score = cosine_similarity([embedding], consensus_embeddings).mean()
        peer_rank_score = from PeerReviewResult (default 0.5 if review failed)
        trust_score = round(semantic_score * 0.6 + peer_rank_score * 0.4, 3)
        is_outlier = (DBSCAN label != consensus_label)
        is_hallucinating = trust_score < 0.45 AND is_outlier
        (BOTH conditions must be true to flag — reduces false positives)
- Step 5: consensus_ratio = len(models_in_consensus_cluster) / len(all_successful)
        low_consensus = consensus_ratio < 0.5
- Return: DetectionResult(trusted, outliers, trust_scores, consensus_ratio, low_consensus)
- API calls: 0 — runs fully local on CPU

STAGE 5 — RESULT COMBINER (agents/combiner.py)
- Input: DetectionResult (uses only trusted responses)
- If low_consensus == True:
    Skip synthesis. Return FinalOutput with answer=None, low_consensus=True
    UI will show warning and all individual responses
- If enough trusted responses:
    Build synthesis prompt:
    "You are a synthesis agent. Below are {n} responses from different AI models
     that broadly agree on the answer. Combine them into one accurate, complete
     answer. Remove redundancy. Preserve important nuances. Do not add information
     not present in the responses below. Write clearly and concisely.

     Responses to synthesize:
     {chr(10).join(f'- {r.response}' for r in trusted)}"
    Call fastest available trusted model (try Groq first, then Gemini, then any)
    Generate 3 follow-up questions with one more quick LLM call
- Return: FinalOutput(answer, consensus_ratio, trust_scores, peer_rankings,
                      hallucination_flags, follow_up_questions, low_consensus,
                      all_results, total_latency_ms)
- API calls: 1 synthesis + 1 follow-up = 2

## DATA MODELS (core/result.py)
```

DATA MODELS (core/result.py)

Implement all these dataclasses exactly:

```python
from dataclasses import dataclass, field
from enum import Enum

class LLMStatus(Enum):
    SUCCESS = "success"
    FAILED  = "failed"
    SKIPPED = "skipped"

@dataclass
class LLMResult:
    model: str
    status: LLMStatus
    response: str | None
    error_type: str | None
    error_msg: str | None
    latency_ms: int
    tokens_used: int = 0
    peer_rank_score: float = 0.5
    trust_score: float = 0.0
    is_outlier: bool = False

@dataclass
class EnhancedQuery:
    original: str
    enhanced: str
    query_type: str

@dataclass
class PeerReviewResult:
    model: str
    raw_ranking_text: str
    parsed_ranking: list
    peer_rank_score: float
```

```
@dataclass
class DetectionResult:
    trusted: list
    outliers: list
    trust_scores: dict
    consensus_ratio: float
    low_consensus: bool

@dataclass
class FinalOutput:
    answer: str | None
    consensus_ratio: float
    trust_scores: dict
    peer_rankings: dict
    hallucination_flags: list
    follow_up_questions: list
    low_consensus: bool
    all_results: list
    total_latency_ms: int
```

LLM ADAPTER INTERFACE (core/adapter.py)

Abstract base class every adapter must implement:

```python
from abc import ABC, abstractmethod
from core.result import LLMResult

class LLMAdapter(ABC):
    @abstractmethod
    async def call(self, prompt: str, image_b64: str | None = None) -> LLMResult:
        pass

    @abstractmethod
    def supports_images(self) -> bool:
        pass
```

```
    @abstractmethod
    def supports_images(self) -> bool:
        pass

    @abstractmethod
    def get_model_id(self) -> str:
        pass

    @property
    @abstractmethod
    def name(self) -> str:
        pass
```

ALL 7 LLM ADAPTERS

Implement one adapter file per provider. All follow the same pattern:
- Constructor reads API key from environment variable
- async def call() implements the API call and returns LLMResult
- Full exception handler (see Exception Handler section below)
- Never raises — always returns LLMResult

Provider details:
1. Groq (agents/groq_adapter.py):
   - Library: from groq import AsyncGroq
   - Key env var: GROQ_API_KEY
   - Default model: llama-3.3-70b-versatile
   - Enhancer model: llama-3.1-8b-instant
   - supports_images: False

2. Gemini (agents/gemini_adapter.py):
   - Library: import google.generativeai as genai
   - Key env var: GEMINI_API_KEY
   - Default model: gemini-2.5-flash
   - supports_images: True (send base64 image in content parts)

3. Cerebras (agents/cerebras_adapter.py):
   - Library: from cerebras.cloud.sdk import AsyncCerebras

```
   - Default model: gemini-2.5-flash
   - supports_images: True (send base64 image in content parts)

3. Cerebras (agents/cerebras_adapter.py):
   - Library: from cerebras.cloud.sdk import AsyncCerebras
   - Key env var: CEREBRAS_API_KEY
   - Default model: llama-3.3-70b
   - supports_images: False

4. Mistral (agents/mistral_adapter.py):
   - Library: from mistralai import AsyncMistral
   - Key env var: MISTRAL_API_KEY
   - Default model: mistral-small-latest
   - supports_images: False

5. OpenRouter (agents/openrouter_adapter.py):
   - Library: from openai import AsyncOpenAI (OpenAI-compatible endpoint)
   - Base URL: https://openrouter.ai/api/v1
   - Key env var: OPENROUTER_API_KEY
   - Default model: deepseek/deepseek-r1:free
   - Second model: meta-llama/llama-4-maverick:free
   - supports_images: False

6. NVIDIA NIM (agents/nvidia_nim_adapter.py):
   - Library: from openai import AsyncOpenAI (OpenAI-compatible endpoint)
   - Base URL: https://integrate.api.nvidia.com/v1
   - Key env var: NVIDIA_NIM_API_KEY
   - Default model: deepseek-ai/deepseek-v4-pro
   - supports_images: False

7. Cohere (agents/cohere_adapter.py):
   - Library: from cohere import AsyncClient
   - Key env var: COHERE_API_KEY
   - Default model: command-r-plus
   - supports_images: False

## EXCEPTION HANDLER (implement in every adapter)

Every adapter's call() method must handle these exact cases:
```

EXCEPTION HANDLER (implement in every adapter)

Every adapter's call() method must handle these exact cases:

```python
import asyncio
import time

async def call(self, prompt: str, image_b64=None) -> LLMResult:
    start = time.time()
    elapsed = lambda: int((time.time() - start) * 1000)

    for attempt in range(2):  # max 2 attempts
        try:
            response_text = await self._make_api_call(prompt, image_b64)

            if not response_text or len(response_text.strip().split()) < 20:
                return LLMResult(self.name, LLMStatus.FAILED, None,
                    "empty_response", "Response too short or empty", elapsed())

            return LLMResult(self.name, LLMStatus.SUCCESS,
                response_text.strip(), None, None, elapsed())

        except Exception as e:
            err_str = str(e).lower()

            # Auth errors — fail immediately, no retry
            if any(x in err_str for x in ['401', '403', 'invalid api key', 'authentication']):
                return LLMResult(self.name, LLMStatus.FAILED, None,
                    "auth", f"Authentication failed: {str(e)}", elapsed())

            # Rate limit — retry with backoff
            if any(x in err_str for x in ['429', 'rate limit', 'quota']):
                if attempt == 0:
                    await asyncio.sleep(2 ** attempt)
                    continue
                return LLMResult(self.name, LLMStatus.FAILED, None,
                    "rate_limit", "Rate limit exceeded", elapsed())
```

```
                "rate_limit", "Rate limit exceeded", elapsed())

        # Timeout
        if isinstance(e, asyncio.TimeoutError):
            return LLMResult(self.name, LLMStatus.FAILED, None,
                "timeout", "Request timed out", elapsed())

        # Server errors — one retry
        if any(x in err_str for x in ['500', '502', '503', '504']):
            if attempt == 0:
                await asyncio.sleep(2)
                continue
            return LLMResult(self.name, LLMStatus.FAILED, None,
                "server_error", f"Server error: {str(e)}", elapsed())

        # Network errors
        if any(x in err_str for x in ['connection', 'ssl', 'network', 'dns']):
            return LLMResult(self.name, LLMStatus.FAILED, None,
                "network", f"Network error: {str(e)}", elapsed())

        # All other errors
        return LLMResult(self.name, LLMStatus.FAILED, None,
            type(e).__name__, str(e), elapsed())

    return LLMResult(self.name, LLMStatus.FAILED, None,
        "max_retries", "Max retries exhausted", elapsed())
```

STREAMLIT UI (app.py and ui/ components)

Build a clean, psychologically-designed Streamlit UI with three states:

STATE A — LANDING: Show only logo + tagline "Ask anything. We ask everyone." + search bar
- st.text_area for text input (2000 char limit with counter)
- Three small buttons below: Text | Voice | Image (switch input mode)
- Voice mode: use streamlit-audio-recorder, send audio to Groq Whisper API for transcription
- Image mode: st.file_uploader + st.camera_input side by side

STATE B — RUNNING: Show enhanced query diff + live model status row + result cards as they arrive

```
STATE B — RUNNING: Show enhanced query diff + live model status row + result cards as they arrive
- Enhanced query: show original (muted) above, enhanced (primary) below
- Model status: horizontal row of model name pills, spinner → ✓ or ✗ as each completes
- Cards fade in as each model finishes — do not wait for all

STATE C — RESULTS: Show consensus answer first, then everything else
- ConsensusAnswerCard: large card, confidence % badge, "Based on N/M models" sub-label
- HallucinationWarningBanner: shown only when consensus_ratio < 0.5, amber color
- st.bar_chart of trust scores (green/amber/red by threshold)
- Per-model cards (st.expander, collapsed by default): response text, trust score bar, latency, peer rank
- Debate mode toggle: show/hide raw peer review critique text
- Three follow-up question buttons (click to run as new query)
- Feedback: thumbs up / thumbs down buttons
- Summary footer: "7 models queried · 5 consensus · 2 flagged · 14.2s"

SIDEBAR:
- Fast mode / Deep mode toggle (fast = 3 models no peer review, deep = all + review)
- Query history (last 20, clickable)
- Model health status (green/red dot per model)

## ADMIN DASHBOARD (pages/admin.py)

Password-gated page. Read ADMIN_PASSWORD_HASH from st.secrets, compare with bcrypt.

Show:
- 4 metric cards: total queries, average confidence %, cache hit rate %, model error rate %
- Filterable log table from SQLite: columns ts, level, event, model, latency_ms, trust_score
- Per-model line chart showing trust score over last 50 queries
- Model health table: name, enabled, queries_today, failure_rate, last_seen

## STRUCTURED LOGGER (core/logger.py)

Implement a logger that writes to both:
1. Rotating flat file: logs/veritasai.log (10MB max, 5 backups)
2. Encrypted SQLite: logs/admin.db using sqlcipher3

Log schema (every field):
ts, level, session_id, event, model, query_hash (SHA-256), latency_ms,
```

2. Encrypted SQLite: logs/admin.db using sqlcipher3

Log schema (every field):
ts, level, session_id, event, model, query_hash (SHA-256), latency_ms,
trust_score, consensus_ratio, error_type, error_msg, tokens_used,
hallucination_flagged (bool), cache_hit (bool), component

Key log events to emit:
- app_start: on Streamlit app initialization
- query_received: when user submits
- query_enhanced: after enhancer completes (include latency)
- cache_hit / cache_miss: before dispatcher
- llm_call_success / llm_call_failed: per model per stage
- rate_limit_hit: when 429 received
- peer_review_complete: after stage 3
- consensus_computed: after detector (include consensus_ratio)
- hallucination_flagged: per model flagged
- synthesis_complete: after combiner
- query_complete: end of pipeline (include total_latency_ms)
- user_feedback: when thumbs up/down clicked

SECURITY REQUIREMENTS

1. All API keys loaded from environment: os.environ.get("KEY_NAME")
   Fall back to st.secrets["KEY_NAME"] for Streamlit Cloud
   Never hardcode any key anywhere in any file

2. DB encryption: use sqlcipher3 instead of sqlite3
   Key: DB_ENCRYPTION_KEY from environment
   Connection: conn = sqlcipher3.connect("logs/admin.db"); conn.execute(f"PRAGMA key='{key}'")

3. Admin password: compare using bcrypt.checkpw()
   import bcrypt; stored_hash = st.secrets["admin_password_hash"].encode()
   bcrypt.checkpw(entered_password.encode(), stored_hash)

4. Input sanitizer must prevent prompt injection:
   Replace or escape: {, }, [,], any string that looks like a jinja template
   Cap length at 2000 chars
   Strip leading/trailing whitespace

```
   Strip leading/trailing whitespace

5. Session rate limiting:
   Store query_count and window_start in st.session_state
   If query_count >= 10 in last 3600 seconds, show "Rate limit reached" and block

## CACHE (core/cache.py)

Session cache using dict in st.session_state:
- Key: SHA-256 hash of enhanced query text
- Value: FinalOutput object
- On cache hit: return cached result, log cache_hit event, skip all API calls

Rate tracker (core/rate_tracker.py):
- JSON file: logs/rate_tracker.json
- Structure: {"groq": {"date": "2026-06-06", "count": 47}, ...}
- On each successful call: increment count
- On new day: reset count to 0
- Before dispatch: check count < model's daily_limit from config.yaml

## CONFIG LOADING

Load config.yaml at startup using PyYAML.
Load .env using python-dotenv.
Build list of enabled LLMAdapter instances from config.
Store in st.session_state to avoid reloading on every Streamlit rerun.

## REQUIREMENTS.TXT

Generate a complete requirements.txt with these packages pinned:
streamlit==1.45.0
streamlit-audio-recorder==0.0.8
aiohttp==3.9.5
groq==0.11.0
google-generativeai==0.8.3
mistralai==1.0.3
cohere==5.5.3
openai==1.35.0
cerebras_cloud_sdk==1.0.0
```

```
google-generativeai==0.8.3
mistralai==1.0.3
cohere==5.5.3
openai==1.35.0
cerebras_cloud_sdk==1.0.0
sentence-transformers==2.7.0
scikit-learn==1.5.0
numpy==1.26.4
pandas==2.2.2
pyyaml==6.0.1
python-dotenv==1.0.1
bcrypt==4.1.3
sqlcipher3==0.5.3
cryptography==42.0.8
colorlog==6.8.2
httpx==0.27.0
langdetect==1.0.9
pyperclip==1.8.2
pytest==8.2.2
pytest-asyncio==0.23.7

## README.md CONTENT

Include sections:
- What is VeritasAI (1 paragraph)
- Architecture overview (the 5 stages in bullet points)
- Setup: clone repo, create .env from .env.example, fill in API keys
- Where to get each API key (with direct signup URLs)
- Local run: pip install -r requirements.txt && streamlit run app.py
- Streamlit Cloud deploy: fork repo, add secrets in Streamlit dashboard, deploy
- Admin dashboard: how to set admin password using bcrypt

## FINAL INSTRUCTIONS

1. Write complete, working, production-quality code for every file
2. Every function must have a docstring
3. Use type hints throughout
4. No placeholder comments like "# TODO" or "# implement this"
5. All error cases handled — the app must never crash from an LLM failure
```

```
httpx==0.27.0
langdetect==1.0.9
pyperclip==1.8.2
pytest==8.2.2
pytest-asyncio==0.23.7

## README.md CONTENT

Include sections:
- What is VeritasAI (1 paragraph)
- Architecture overview (the 5 stages in bullet points)
- Setup: clone repo, create .env from .env.example, fill in API keys
- Where to get each API key (with direct signup URLs)
- Local run: pip install -r requirements.txt && streamlit run app.py
- Streamlit Cloud deploy: fork repo, add secrets in Streamlit dashboard, deploy
- Admin dashboard: how to set admin password using bcrypt

## FINAL INSTRUCTIONS

1. Write complete, working, production-quality code for every file
2. Every function must have a docstring
3. Use type hints throughout
4. No placeholder comments like "# TODO" or "# implement this"
5. All error cases handled — the app must never crash from an LLM failure
6. The Streamlit app must handle async code correctly using asyncio.run()
   or nest_asyncio for the Streamlit environment
7. Use st.session_state to persist adapters, cache, and session data
8. The admin dashboard must be in pages/admin.py (Streamlit multipage)
9. Write 4 test files covering: detector logic, dispatcher behavior,
   peer review parsing, and input sanitizer edge cases
10. The app must work with as few as 1 API key configured
    (disable missing-key models gracefully)
```

End of VeritasAI Master Plan — Version 1.0
Generated from full planning session. Ready for development.